

GUIA- CONCURRENCIA - TRANSACCIONES

Ejercicio 5.1

Considere el siguiente solapamiento de dos transacciones T1 y T2 que operan sobre los ítems de datos X e Y.

$b_{T1}; R_{T1}(X); b_{T2}; W_{T1}(X); R_{T2}(X); R_{T1}(Y); W_{T2}(X); W_{T1}(Y); c_{T1}; c_{T2};$

- Indique si el solapamiento es recuperable, justificando su respuesta.
- Indique si podrían ocurrir *rollbacks* en cascada ante un eventual aborto de una transacción en medio del solapamiento, justificando su respuesta.

a. $b_{T1}; R_{T1}(X); b_{T2}; W_{T1}(X); R_{T2}(X); R_{T1}(Y); W_{T2}(X); W_{T1}(Y); c_{T1}; c_{T2};$

T2 lee un X que lo grabo T1 pero T2 commitea despues que T1

El solapamiento es recuperable

$R_{T2}(X)$ es lectura de dato escrito por otra transaccion, pero como T1 hace commit antes que T2, la transaccion T2 NO confirma datos leídos de una transaccion NO confirmada. Por lo tanto es recuperable

b. Si, permite rollback en cascada

En $W_{T1}(X)$ y $R_{T2}(X)$.

T2 lee X escrito por T1 antes de que T1 haga commit. Aunque se agreguen los commits al final, las lecturas ya ocurrieron antes de esos commits. Si T1 abortara antes de commitear, T2 tendria que abortar porque leyó datos no confirmados produciendo un rollback en cascada

Ejercicio 5.2

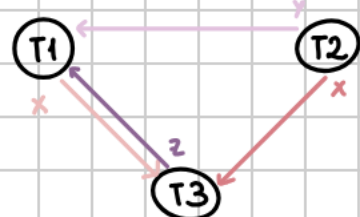
Dado los siguientes solapamiento de transacciones:

- $b_{T1}; R_{T1}(X); b_{T2}; R_{T2}(X); W_{T2}(Y); b_{T3}; W_{T3}(Z); R_{T1}(Z); W_{T3}(X); R_{T1}(Y);$
- $b_{T1}; b_{T2}; R_{T1}(X); R_{T2}(X); b_{T3}; W_{T3}(Y); W_{T3}(Z); W_{T1}(Y); W_{T2}(X); R_{T2}(Z);$

Conteste de cada uno las siguientes preguntas:

- Indique, con una justificación mínima, si el solapamiento es serializable. En caso de serlo, indique un orden de transacciones equivalente.
- Agregue, luego de las operaciones, los commits de todas las transacciones para que el solapamiento sea recuperable. En caso de no ser posible, justificarlo.
- Indique si el solapamiento con los commits agregados permite o evita *rollbacks* en cascada.

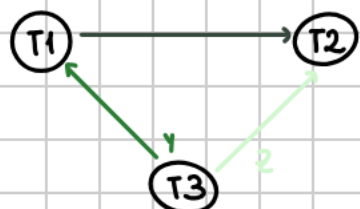
1 a.



- Conflicto lectura y escritura
- Conflicto lectura y escritura
- Conflicto escritura y lectura
- Conflicto escritura y lectura

Al haber ciclos, el grafo de precedencias **NO** es serializable.

2 a.



- Conflicto escritura y escritura
- Conflicto escritura y lectura
- Conflicto lectura y escritura

Al **NO** haber ciclos, el grafo de precedencias es serializable. Un orden de transacciones validas: T3-T1-T2

1b. $b_{T1}; R_{T1}(x); b_{T2}; R_{T2}(x); \underline{W_{T2}(y)}; b_{T3}; \underline{W_{T3}(z)}; R_{T1}(z); \underline{W_{T3}(x)}; T_1(y); C_{T2}; C_{T3}; C_{T1}$

T_1 lee un z que lo grabo T_3 pero T_1 committea despues que T_3

El solapamiento es recuperable

$R_{T1}(z)$ es una lectura de datos escritos por otra transaccion. Pero como T_3 hace commit antes que T_1 , ninguna transaccion confirma datos leidos de una transaccion NO confirmada, Por lo tanto es recuperable

2b. $b_{T1}; b_{T2}; R_{T1}(x); R_{T2}(x); b_{T3}; \underline{W_{T3}(y)}; \underline{W_{T3}(z)}; \underline{W_{T1}(y)}; \underline{W_{T2}(x)}; R_{T2}(z); C_{T1}; C_{T3}; C_{T2}$

T_2 lee un z que lo grabo T_3 pero T_2 committea despues que T_3

El solapamiento es recuperable

$R_{T2}(z)$ es una lectura de datos escritos por otra transaccion. Pero como T_3 hace commit antes que T_2 , ninguna transaccion confirma datos leidos de una transaccion NO confirmada, Por lo tanto es recuperable

1c.

. En $W_{T3}(z); \dots; R_{T1}(z)$

T_1 lee z escrito por T_3 antes de que T_3 haga commit

Aunque se agreguen los commits al final, las lecturas ya ocurrieron antes de esos commits

Si T_3 aborta antes de committear, T_1 tendria que abortar porque leyo datos no confirmados

Produciendo un rollback en cascada

Si, permite rollback en cascada, no evita porque lee datos escritos por otra transaccion que no hizo commit

2c

En $W_{T3}(z); \dots; R_{T2}(z)$

T_2 lee z escrito por T_3 antes de que T_3 haga commit

Aunque se agreguen los commits al final, las lecturas ya ocurrieron antes de esos commits

Si T_3 aborta antes de committear, T_2 tendria que abortar porque leyo datos no confirmados

Produciendo un rollback en cascada

Si, permite rollback en cascada, no evita porque lee datos escritos por otra transaccion que no hizo commit

GUIA- RECUPERACION

Ejercicio 6.1

Un SGBD implementa el algoritmo de recuperación UNDO/REDO. Para la siguiente secuencia de registros de *log*:

- Indique qué items deben/pueden haber cambiado su valor en disco.
- indique cómo queda el archivo de *log*.

```
01 (BEGIN, T1);  
02 (WRITE, T1, A, 20, 80);  
03 (BEGIN, T2);  
04 (WRITE, T2, B, 30, 44);  
05 (WRITE, T1, C, 44, 23);  
06 (WRITE, T2, D, 14, 12);  
07 (COMMIT, T2);
```

Se recorre el log de atras hacia adelante

Por cada transaccion de la que no hay commit, se deshacen sus operaciones restaurando el valor anterior correspondiente a cada WRITE encontrado (UNDO)

Se encuentra 07 (COMMIT, T2) así que es una transaccion con commit, recorriendo hacia atras T1 no tiene commit siendo una transaccion sin commit y debe deshacerse

Se aplica UNDO sobre T1 restaurando el valor de C; C = 44. Tambien se restaura por T1 A = 20

Despues se recorre el log de adelante hacia atras reaplicando las escrituras de las transacciones que hicieron commit (REDO)

Como T2 hizo commit se rehace quedando B = 44; y tambien D = 12.

Para cada transaccion que no realizo commit, se agrega un registro de aborto al log y se fuerza su escritura a disco

01 (BEGIN, T1);	A = 20
02 (WRITE, T1, A, 20, 80);	B = 44
03 (BEGIN, T2);	C = 44
04 (WRITE, T2, B, 30, 44);	D = 12
05 (WRITE, T1, C, 44, 23);	
06 (WRITE, T2, D, 14, 12);	
07 (COMMIT, T2);	
08 (ABORT, T1);	

Ejercicio 6.2

Un SGBD implementa el algoritmo de recuperación UNDO con *checkpoint* activo. Luego de una falla, el sistema encuentra el siguiente archivo de *log*:

```
01 (BEGIN, T1);
02 (BEGIN, T2);
03 (WRITE T1, X, 18);
04 (WRITE T2, Y, 4);
05 (BEGIN, T3);
06 (WRITE T1, Z, 20);
07 (COMMIT, T1);
08 (WRITE T2, Z, 6);
09 (BEGIN CKPT, T2, T3);
10 (WRITE T3, X, 0);
11 (COMMIT, T2);
12 (WRITE T3, Z, 8);
13 (BEGIN, T4);
14 (WRITE T4, Y, 0);
```

Explique cómo se llevará a cabo el procedimiento de recuperación, indicando qué cambios deben ser realizados en disco y en el archivo de *log*.

Como el checkpoint es incompleto, no alcanza con empezar desde el BEGIN CKPT, se recorre todo el log ; voy a deshacer las TNU, deshago T3 restaurando el valor anterior de Z; Z=8. Deshago T4 restaurando el valor de Y; Y=0. Se hace flush de esos datos a disco. Agrego 15 (ABORT, T3), 16 (ABORT, T4) en el log y hago flush del log al disco.

Ejercicio 6.3

Un SGBD implementa el algoritmo de recuperación REDO con *checkpoint* activo. Luego de una falla, el sistema encuentra el siguiente archivo de *log*:

```
01 (BEGIN, T1);
02 (WRITE T1, A, 15);
03 (BEGIN, T2);
04 (WRITE T1, B, 38);
05 (COMMIT, T1);
06 (WRITE T2, B, 30);
07 (BEGIN CKPT, T2);
08 (BEGIN, T3);
09 (WRITE T3, C, 40);
10 (COMMIT, T2);
11 (BEGIN, T4);
12 (WRITE T4, D, 55);
13 (END CKPT);
14 (COMMIT, T4);
```

Explique:

- cómo se llevará a cabo el procedimiento de recuperación.
- indicando hasta qué punto del archivo de *log* se deberá retroceder.
- qué cambios deberán ser realizados en disco y en el archivo de *log*.

en disco.

Se rehacen las demás que hicieron commit, T2 y T4.

Se rehace T2 escribiendo B=30; se rehace T4 escribiendo D=55. Se hacen flush de estos datos a disco.

La T3 al no haber hecho commit, escribo 15 (ABORT, T3) en el archivo del log y se hace flush del log al disco.

El procedimiento se llevara:

Comienzo desde abajo hacia arriba, encuentro

13 (BEGIN, T4), T4 no commito entonces es TNU (transacción no válida), encuentro

09 (BEGIN CKPT, T2, T3), T2 es TV (transacción válida) por commitar, en cambio T3 es TNU continuo y T1 también es TV.

Transacciones válidas = {T1, T2}

Transacciones no válidas = {T3, T4}

A las TNU hay que abortarlas, las TV permanecen.

El proceso de recuperación se llevara:

Voy de arriba hacia abajo.

Busco el último checkpoint completo, lo encuentro desde la línea 07 (BEGIN CKPT, T2) a 13 (END CKPT) haciéndolo completo.

Transacciones en el checkpoint = {T2},

Busco su BEGIN que está en la línea 03 (BEGIN, T2).

Voy a retroceder hasta ese punto.

Transacciones con commit = {T1, T2, T4}

Transacciones sin commit = {T3}

Como el checkpoint es completo se asegura que como T1 hizo commit antes del BEGIN CKPT sus cambios ya están asegurados

GUIA: COSTOS

Hecha en la Practica y en finales

GUIA : NO SQL

8.1

Una cadena de supermercados guarda sus ventas en MongoDB. Cada documento representa una venta e incluye: número de ticket (usado como `_id`), fecha y hora, DNI y nombre del comprador, un array de productos (cada uno con código, nombre, cantidad y precio), y el monto total.

```
json
{
  "_id": "0057-46290015",
  "fecha_venta": Date("2018-02-06 11:53:28"),
  "DNI_comprador": 38115218,
  "nombre_comprador": "BENITEZ, JUAN ANTONIO",
  "productos": [
    { "codigo_producto": 108431, "nombre_producto": "pañales makako 20 un.", "cantidad": 1, "precio": 282.50 },
    { "codigo_producto": 210160, "nombre_producto": "papas fritas boom 150gr.", "cantidad": 1, "precio": 0.50 },
  ],
  "monto_total": 282.50
}
```

Encontrá los 5 productos que más frecuentemente se compran en el mismo ticket que pañales Makako (codigo_producto 108431), indicando cuántos tickets distintos incluyen cada producto junto con los pañales. El resultado debe tener este formato:

```
json
{ "_id": 210160, "cantidad_tickets_compra": 3804 }
```

```
db.collection.aggregate([
  {
    $match: {
      "productos.codigo_producto": {$in: [108431]} // Me fijo los productos que
    } // en su ticket tengan a los pañales
  },
  {
    $unwind: "$productos" // Descompongo por productos
  },
  {
    $match: {
      "productos.codigo_producto": {$ne: 108431} // Descarto los productos que sean pañales
    }
  },
  {
    $group: {
      _id: "$productos.codigo_producto", // AGrupe por producto para saber en cuantos
      cant_tickets: {$sum: 1} // tickets aparecen
    }
  },
  {
    $sort: {cant_tickets: -1} // Ordeno de mayor a menor por aparicion
  },
  { $limit: 5 }, // Me quedo con los 5 mas pedidos
  {
    $project: { // Formateo la respuesta a la pedida
      _id: "$_id",
      cantidad_tickets_compra: "$cant_tickets"
    }
  }
])
```

8.2

La Facultad de Ingeniería tiene una colección `Graduados` en MongoDB donde cada documento representa un alumno y tiene esta estructura:

```
{
  "_id": ObjectId("207ca8372c83b6b092d12512"),
  "padron": 97294,
  "apellido": "Martinez",
  "nombres": "Gonzalo",
  "notas": [
    {
      "codigomateria": 7541,
      "nombremateria": "ALGORITMOS Y PROGRAMACION II",
      "notafinal": 8
    },
    {
      "codigomateria": 6620,
      "nombremateria": "ORGANIZACION DE COMPUTADORAS",
      "notafinal": 9
    }
  ]
}
```

Objetivo: obtener un histograma que diga, para cada nota promedio posible (truncada a un decimal), cuántos alumnos egresan con esa nota.

json

```
{ "notapromedio": 7.7, "cantidadalumnos": 184 }
{ "notapromedio": 7.8, "cantidadalumnos": 240 }
```

Pista que da la cátedra: como `$trunc` no permite especificar decimales directamente, el truco para truncar a un decimal es dividir por 0.1, truncar, y volver a multiplicar por 0.1. Igual que en el ejemplo del padrón pero con 10 en vez de 10000.

```
db.collection.aggregate([
  {
    $unwind: "$notas" // Descompongo por notas
  },
  {
    $group: {
      _id: "$padron", // Agrupo por promedio de alumno
      promedio: {
        $avg: "$notas.notafinal" y alumno
      }
    }
  },
  {
    $project: {
      promedio: { $multiply: [
        { $trunc: { $divide: ["$promedio", 0.1] } }, // Trunco el promedio
        0.1 dada por la cátedra
      ] }
    }
  },
  {
    $group: {
      _id: "$promedio", // Agrupo por promedio
      cant_alumnos: { $sum: 1 } para ver cuantos alumnos tienen ese promedio
    }
  },
  {
    $project: {
      _id: 0, // Formato la salida
      nota.promedio: "$_id",
      cantidadalumnos: "$cant_alumnos"
    }
  }
])
```